Section 5 : Structure of a C++ Program

Informations sur le papier

- Titre : `Programmation C++`
- Auteurs : `Etienne KOA`

Aperçu de la section

- La structure d'un programme `C++`.
  - Composants de base
  - Directives du préprocesseur
  - La fonction principale
  - Espaces de noms
  - Commentaires
  - E/S de base

Les mots-clés C++

`#include` Directive du préprocesseur

- Qu'est-ce qu'un préprocesseur ?
- Ça fait quoi ?
- Les directives commencent par `#`
- Commandes au préprocesseur

La fonction principale()

- Chaque programme `C++` doit avoir exactement 1 fonction `main()`.
- Point de départ de l'exécution du programme.
- return 0 indique une exécution réussie du programme.
- 2 versions toutes deux valables

Espaces de noms

- Pourquoi `std::cout` et pas seulement `cout` ?
- Qu'est-ce qu'un conflit de nom ?
- Noms donnés aux parties du code pour aider à réduire les conflits de noms.
- `std` est le nom de l'espace de noms `C++ standard`.
- Les frameworks tiers auront leurs propres espaces de noms.
- Opérateur de résolution de portée `::`
- Comment utiliser ces espaces de noms ?

A - C	D - P	R - Z
alignas (C++11)	decltype (C++11)	reflexpr (reflection TS)
alignof (C++11)	default (1)	register (2)
and	delete (1)	reinterpret_cast
and_eq	do	requires (C++20)
asm	double	return
atomic_cancel (TM TS)	dynamic_cast	short
atomic_commit (TM TS)	else	signed
atomic_noexcept (TM TS)	enum (1)	sizeof (1)
auto (1) (2) (3) (4)	explicit	static
bitand	export (1) (3)	static_assert (C++11)
bitor	extern (1)	static_cast
bool	false	struct (1)
break	float	switch
case	for (1)	synchronized (TM TS)
catch	friend	template
char	goto	this (4)
char8_t (C++20)	if (2) (4)	thread_local (C++11)
char16_t (C++11)	inline (1)	throw
char32_t (C++11)	int	true
class (1)	long	try
compl	mutable (1)	typedef
concept (C++20)	namespace	typeid
const	new	typename
consteval (C++20)	noexcept (C++11)	union
constexpr (C++11)	not	unsigned
constinit (C++20)	not_eq	using (1)
const_cast	nullptr (C++11)	virtual
continue	operator (4)	void
co_await (C++20)	or	volatile
co_return (C++20)	or_eq	wchar_t
co_yield (C++20)	private (3)	while
	protected	xor
	public	xor_eq

Figure 1: Mots clés C++

```
#include <iostream>
#include "myfile.h"

#if
#elif
#else
#endif
```

```
#ifdef
#ifndef
#define
#undef

#line
#error
#pragma
```

BEGINNING C++ PROGRAMMING
Preprocessor Directives

Figure 2: Mots clés C++

```
int main()
{
   // code

   return 0;
}
```

```
int main(int argc, char *argv[])
{
   // code

   return 0;
}
```

```
program.exe
```

```
program.exe argument1 argument2
```

BEGINNING C++ PROGRAMMING
Main Function

Figure 3: fonction principale()

```
#include <iostream>

int main()
{
    int favorite_number;

    std::cout << "Enter your favorite number between 1 and 100: ";

    std::cin >> favorite_number;

    std::cout << "Amazing!! That's my favorite number too!" << std::endl;
    std::cout << "No really!!, " << favorite_number
              << " is my favorite number!" << std::endl;

    return 0;
 }
```

Figure 4: Explicité utilisant les espaces de noms

```
#include <iostream>

using namespace std;     // Use the entire std namespace

int main()
{
    int favorite_number;

    cout << "Enter your favorite number between 1 and 100: ";

    cin >> favorite_number;

    cout << "Amazing!! That's my favorite number too!" << endl;
    cout << "No really!!, " << favorite_number
         << " is my favorite number!" << endl;

    return 0;
 }
```

Figure 5: La directive using namespace

Explicitité à l'aide des espaces de noms

La directive using namespace

Qualifié en utilisant la variante nameapce

```
#include <iostream>

using namespace std;     // Use the entire std namespace

int main()
{
    int favorite_number;

    cout << "Enter your favorite number between 1 and 100: ";

    cin >> favorite_number;

    cout << "Amazing!! That's my favorite number too!" << endl;
    cout << "No really!!, " << favorite_number
         << " is my favorite number!" << endl;

    return 0;
 }
```

Figure 6: Qualifié à l'aide d'une variante d'espaces de noms

E/S de base utilisant `cin` et `cout`

- `cout, cin, cerr` et `clog` sont des objets représentant des flux.
- `cout`
  - flux de sortie standard.
  - console
- `cin`
  - flux d'entrée standard

  - clavier

- `<<`
  - Opérateur d'insertion
  - flux de sortie

- `>>`
  - opérateur d'extraction
  - flux d'entrée

`cout` et `<<`

- Insérez les données dans le flux `cout`

```
cout << données ;
```

- Peut être enchaîné

```
cout << "les données 1 sont " << data1;
```

- N'ajoute pas automatiquement de sauts de ligne

```
cout << "les données 1 sont " << data1 << endl;
cout << "les données 1 sont "<< data1 << "\n";
```

`cin` et `>>`

- Données extraites du flux `cin` en fonction du type de données.

```
cin >> data ;
```

- Peut être enchaîné

```
cin >> data1 >> data2;
```

- Peut échouer si les données saisies ne peuvent pas être interprétées. Les « données » peuvent avoir une valeur indéterminée.